מטלה מסכמת- חלק א: מבוא ללמידת מכונה ומדעי הנתונים

מגישים:

- ליעד מנדיל, 315096073
- מקסים פרוקופצוק, 337875397

תהליך העבודה שלנו:

החלטנו לבנות את המטלה בזרימה הבאה:

קובץ: spaceship_dataEDA.ipynb

1. טעינה ראשונית של הדאטה ובדיקת כל תכונה (עמודה) בנפרד.
2. מציאת ערכים חסרים בדאטה בעזרת הצלבות מידע בין עמודות ומציאת הסתברויות מותנות (עד כמה שנצליח)
3. החלטה על האם למחוק נוסעים בטענה שהמידע שחסר להם לא ניתן להשלמה בלי לפגוע בביצועי המסווגים ונטייה של המידע.
4. הורדה והוספת עמודות חדשות בהתאם לשיקולים שנציג
5. בדיקה סופית של המידע ושמירתו בקובץ final_data.csv

קבצים: decision_Tree.ipynb, GNB.ipynb, KNN.ipynb, LDA.ipynb, Logistic_Regression.ipynb, MLP.ipynb, QDA.ipynb, Random_Forest.ipynb, SVM.ipynb, xgboost.ipynb.

1. בכל אחד מקבצים אלו בנינו את המסווג בעזרת ההייפר פרמטרים האופטימליים לדאטה שלנו לאחר בדיקות קרוס ווליד‌ציה וקיפולי סדרת האימון.

קובץ final.ipynb

1. יצירת המודל הסופי בעלי ההייפר פרמטרים בנבחרים מכל מסווג
2. בנייתם
3. יצירת חיזוי על סדרת המבחן וקבלת אחוז דיוק של כל מסווג על סדרה זאתי
4. השוואה בין התוצאות ומסקנה סופית

EDA:

תהליך קבלת ההחלטות שלנו:

טעינה ראשונית של הדאטה:

בטעינה הראשונית ראינו כי הדאטה מהצורה: (8693,14) כאשר 13 עמודות הן תכונות של כל נוסע ועמודה 14 מצביעה לנו אם הנוסע עבר טרנספורמציה או לא (שעל עמודה זאתי יהיה תהליך הסיווג שלנו).

בדקנו את טיפוסי המשתנים של הדאטה, שכן במסווגים הדאטה חייב להיות מספרי, וראינו שרובו אובייקטי.

בדקנו ייחודיות ראשונית של כל העמודות, על מנת להבין כמה אופציות יש לכל אחד מהעמודות (בלי ולידציה אם הגיוניות עדיין, רק ראשוני).

הצגת דוגמא לנוסעים:

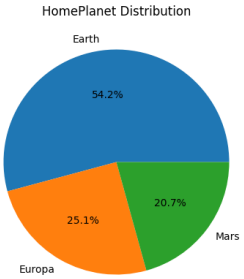
	PassengerId	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck	Name	Transported
0	0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0	0.0	0.0	0.0	0.0	Maham Ofracculy	False
1	0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0	9.0	25.0	549.0	44.0	Juanna Vines	True
2	0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0	3576.0	0.0	6715.0	49.0	Altark Susent	False
3	0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0	1283.0	371.0	3329.0	193.0	Solam Susent	False
4	0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0	70.0	151.0	565.0	2.0	Willy Santantines	True
5	0005_01	Earth	False	F/0/P	PSO J318.5-22	44.0	False	0.0	483.0	0.0	291.0	0.0	Sandie Hinetthews	True
6	0006_01	Earth	False	F/2/S	TRAPPIST-1e	26.0	False	42.0	1539.0	3.0	0.0	0.0	Billex Jacostaffey	True
7	0006_02	Earth	True	G/0/S	TRAPPIST-1e	28.0	False	0.0	0.0	0.0	0.0	NaN	Candra Jacostaffey	True
8	0007_01	Earth	False	F/3/S	TRAPPIST-1e	35.0	False	0.0	785.0	17.0	216.0	0.0	Andona Beston	True
9	0008_01	Europa	True	B/1/P	55 Cancri e	14.0	False	0.0	0.0	0.0	0.0	0.0	Erraiam Flatic	True

חקר + שינוי טיפוסי המשתנים לערכים המתאימים לסיווג:

עברו כל עמודה בנפרד על מנת להכין אותה לערכים מספריים:

HomePlanet

לאחר הצגה של value_count:



HomePlanet	
Earth	4602
Europa	2131
Mars	1759

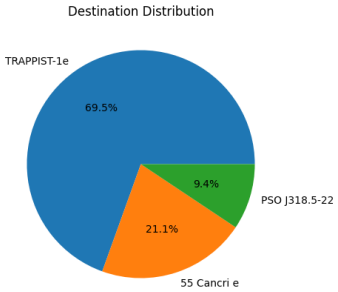
ראינו כי ישנם 3 אופציות לכוכב בית שממנו הגיעו הנוסעים: Earth, Europa, Mars, ושרוב האנשים מגיעים מ Earth.

מאחר והעמודה הזאתי היא מטיפוס string, ואנחנו נצטרך לבצע המרה של השמות למספרים על פה מילון שבחרנו:

```
homeplanet_map = {
    'Earth': 0,
    'Europa': 1,
    'Mars': 2
}
```

Destination:

לאחר הצגה של value_count:



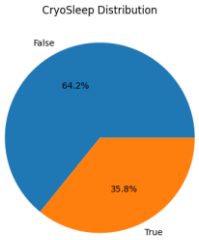
Destination	
TRAPPIST-1e	5915
55 Cancri e	1800
PSO J318.5-22	796

מצאנו כי יש 3 אופציות ליעדים, ומאחר וגם הם כמו כוכב המוצא מטיפוס string, אנחנו נצטרך ליצור מילון שממרים את השמות למספרים:

```
destination_map = {
    'TRAPPIST-1e': 0,
    'PSO J318.5-22': 1,
    '55 Cancri e': 2
}
```

CryoSleep:

לאחר הצגה של value_count:




```
CryoSleep
False    5439
True     3037
```

אפשר לראות ש64 אחוז מהנוסעים (שאין להם ערך NULL בעמודה זאת), החליטו שהם נשארים ערים במהלך הטיסה.

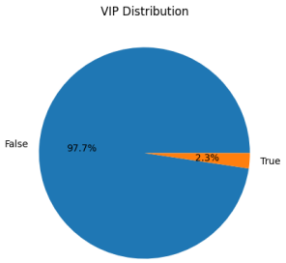
משתנה זה מסוג בוליאני וגם כאן נמיר למשתנה מספרי לשימוש במסווגים בהמשך:

```
spaceship_data_copy['CryoSleep'] = spaceship_data_copy['CryoSleep'].map({True: 1, False: 0}).fillna(-1).astype(int)
```

על מנת למלא את כל המילונים האלו הגדרנו כל מקום שהוא null לשים 1- , ובסוף אנחנ מחזירים את כולם חזרה להיות ערך ריק.

VIP:

לאחר הצגה של value_count:




```
VIP
False    8291
True      199
```

משתנה זה גם הוא בוליאני, וניתן לראות שמרבית הנוסעים אינם VIP , ככל הנראה מידע זה יעזור לנו בהמשך לקבלת ההחלטה של מילוי ערכים חסרים.

גם כאן ביצענו המרה ממשתנה בוליאני למספרי:

```
spaceship_data_copy['VIP'] = spaceship_data_copy['VIP'].map({True: 1, False: 0}).fillna(-1).astype(int)
```

Transported:

בדקנו כי בעמודה זו אין ערכים חסרים, שכן מדובר במשתנה היעד (Target) הנדרש לתהליך הסיווג. ניסיון לאמוד או להשלים ערכים חסרים באופן ידני היה עלול לפגוע בתהליך הלמידה ולהוביל להטיה בתוצאות.

בנוסף, זיהינו כי המשתנה הוא בוליאני (Boolean), ולכן המֵרנו אותו לייצוג מספרי לצורך עבודה נוחה עם אלגוריתמי למידת מכונה.

```
spaceship_data_copy['Transported'] = spaceship_data_copy['Transported'].map({True: 1, False: 0}).fillna(-1).astype(int)
```

לאורך שלב הEDA- לא נשתמש בעמודה זו כחלק מהניתוחים וההסקות, משום ששימוש במשתנה היעד עלול ליצור מצב של *data leakage*("הצצה לתשובה"), כלומר שימוש במידע שאמור להיות מנובא, דבר שעלול להוביל למסקנות מוטות ולפגיעה באמינות המודל.

PassengerId:

דבר ראשון היינו צריכים לבדוק שאין כפילויות בעמודה זאת, שכן זהו משתנה שאמור להיות ייחודי לכל נוסע בנפרד.

לאחר מכן בדקנו את תקינות המספר זהות (מאחר שאנו יודעים כי המספרי זהות האלו צריכים להיראות מהצורה: gggg_pp , אז נעזרנו בLLM על מנת ליצור את הREGEX : '^\d{4}_\d{2}$' , שמוודא לנו שכל תא בעמודה הזאתי הוא מהצורה המתאימה או לא. גילינו כי כולם תקינים.

מאחר שאנחנו רוצים להביא את כל הערכים להיות ערכים מספריים, החלטנו לפצל את העמודה ל2 עמודות חדשות: PassengerId -> GroupId, GroupMemberId מאחר ואנחנו נרצה בהמשך לבצע בדיקות על קבוצות להשלמת ערכים חסרים, ועל מיקומם בקבוצה. ובנוסף גם נוריד את עמודה PassengerId מאחר ו2 העמודות החדשות נותנות לי ממנה את כל המידע הרלוונטי.

Cabin:

בבדיקת הייחודיות מההתחלה גילינו כי יש 6560 תאים בספינה, דבר שנותן לנו את ההבנה שרוב האנשים נוסעים לבד.

יצרנו גם כאן regex לבדיקת התקינות: '^[A-GT]/\d+/(P|S)$' , שעוזר לנו לבדוק את התקינות של התא מהצורה: deck/num/side.

לאחר הבדיקת תקינות, ומהסיבה שאנחנו צריכים לעבוד במספרים החלטנו לפצל את Cabin:

Cabin -> Deck,RoomNum,RoomSide על מנת למספר ושנוכל להשתמש בהמשך במידע .

לdeck ולroomside יצרנו מילונים על מנת להמירם למספרים:

```
deck_map = {
    'A': 0,
    'B': 1,
    'C': 2,
    'D': 3,
    'E': 4,
    'F': 5,
    'G': 6,
    'T': 7
}
```

```
roomside_map = {
    'P': 0,
    'S': 1,
}
```

Name:

בהתחלה בדקנו ייחודיות של שמות האנשים, וגילינו שיש 400 אנשים בלי שם ייחודי.

בהתחלה חשבנו שהאנשים רשומים כפול, אך מאחר וגילינו מקודם כי התעודת זהות של כל בנאדם ייחודי, אז ככל הנראה זה מקרי בלבד ולא בעל חשיבות

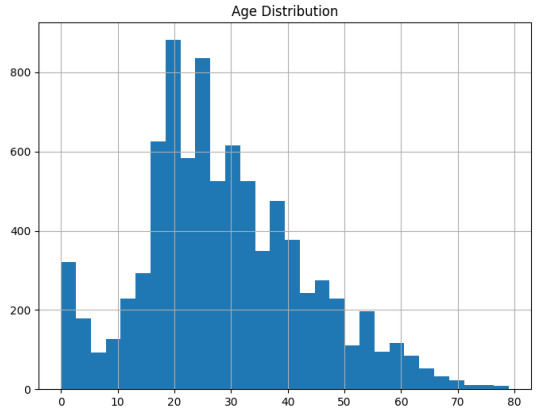
החלטנו לבדיקה לפצל את השמות לשם פרטי ושם משפחה, וזאת על מנת שבהמשך נוכל לבדוק קורלציה בין משפחות.

```
spaceship_data_copy[['FirstName', 'Surname']] = spaceship_data['Name'].str.split(' ', expand=True)
```

כמובן שבדקנו שלכולם יש 2 שמות ואנחנו לא משאירים תאים ריקים.

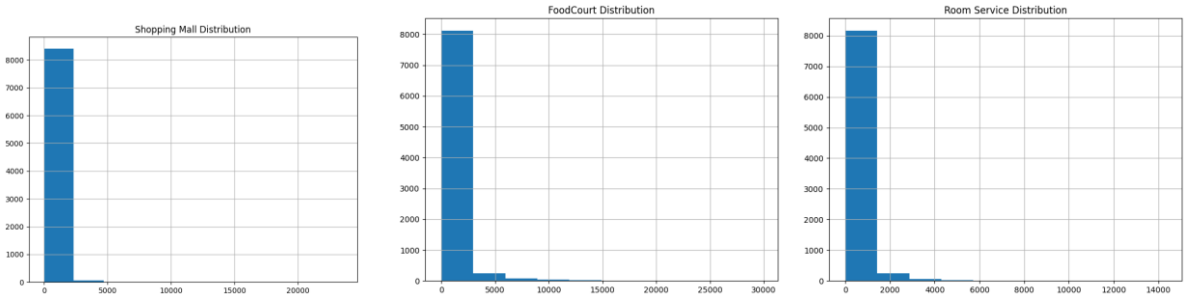
Age:

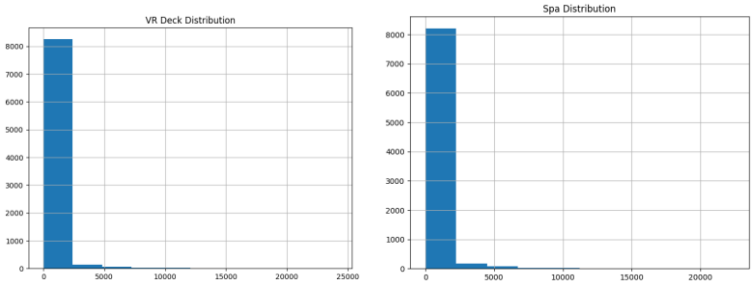
מאחר ומשתנה זה מספרי, והוא כבר מסודר לסיווג, רק הסתכלנו כאן על ההיסטוגרמה ששלו:



RoomService + FoodCourt + ShppingMall + Spa + VRDeck

משתנים אלו משתנים מספריים בלי צורך בבדיקות, רק הצגנו את ההיסטוגרמות:





מציאת ערכים חסרים בדאטה:

בחלק זה נעבור עמודה עמודה, נמצא כמה נוסעים חסר להם ערך מעמודה זאת, וננסה על סמך הסתברויות והצלבת מידע למלא את הנתונים בצורה מושכלת (תהליך שמדמה למידת מכונה ברמה הסביבית, במקום ליצור מסווגים לכל עמודה בנפרד).

בקובץ זה נציג רק את המסקנות המשמעותיות שמצאנו ולא את כל הבדיקות שביצענו.

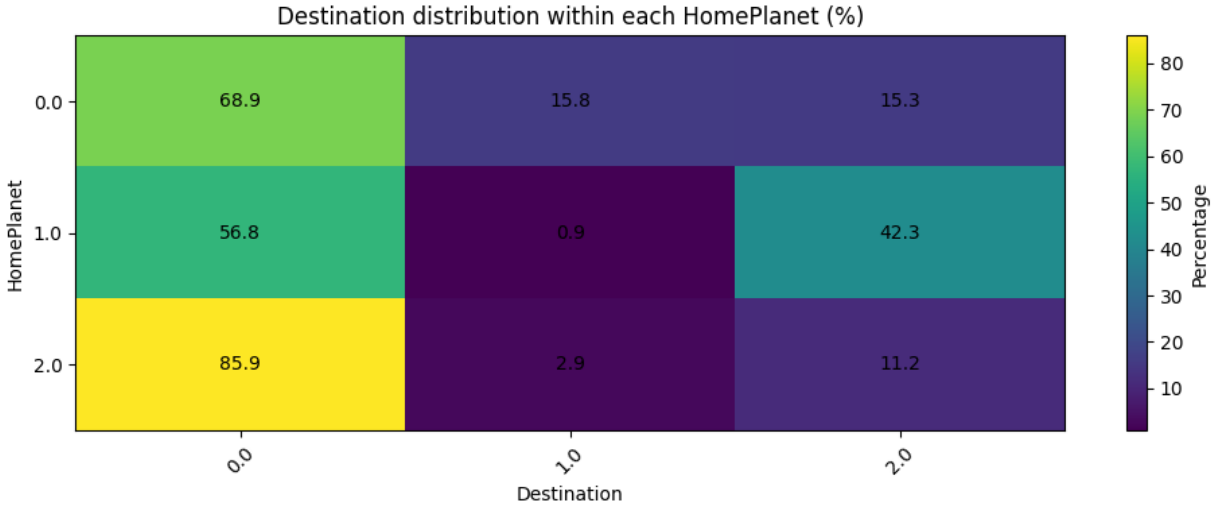
HomePlanet

	Count
HomePlanet is NaN	201
HomePlanet not NaN	8492
Total rows	8693

ישנם 201 נוסעים ללמה כוכב מוצא.

הנתונים שגילינו לאורך הדרך:

- ניתוח התפלגות מותנית של יעד המוצא לפי יעד נסיעה:



מצאנו בבדיקה זו כי ישנו קשר בין כוכב היעד 0 (TRAPPIST-1e), לבין כוכב המוצא 2 (Earth). כלומר 85.9% מהאנשים שהגיעו מכוכב 2 , כוכב היעד שלהם היה 0.

מאחר וההסתברות הזאתי גבוהה נוכל להישען אליה כוכב מוצא חסר בנוסעים שאין להם כוכב מוצא ויש להם כוכב יעד 0.

- <u>מילוי כוכב אם לפי קבוצות:</u>

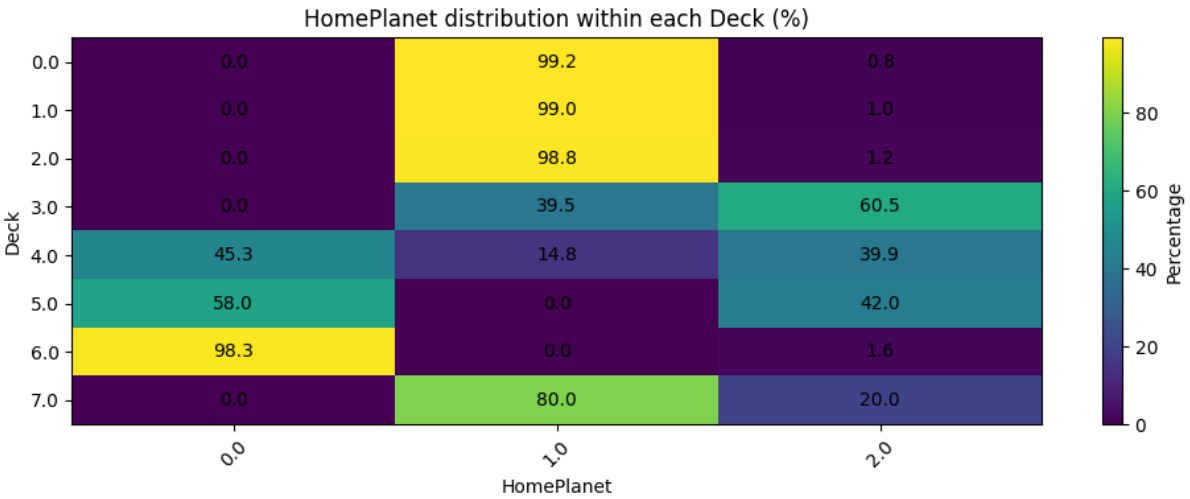
מאחר ופיצלנו בהתחלה את הPassenger_id ל2 עמודות שאחת מהן היא מספר קבוצה, יכולנו ליצור איחוד של כל קבוצה בנפרד ולבחון האם יש להם אותו כוכב אם משותף

מצאנו כי ניתן למלא מספר נוסעים בעזרת בדיקה זאתי .

לקחנו רק את המקרים ש100% מהנוסעים שבקבוצה הגיעו מאותו מקום, מאחר ולא יכולנו להראות קורלציה בין ה2.

	Metric	Value
0	Missing before filling	51.00
1	Values filled	19.00
2	Missing after filling	32.00
3	Fill rate (%)	37.25

- <u>ניתוח התפלגות מותנית של יעד המוצא לפי קומה בספינה:</u>



מצאנו כי יש קשר ישיר:

בקומות 0,1,2 מתאכלסים כמעט בבלעדיות מלאה אנשים שהגיעו מכוכב לכת 1,

בקומה 6, 98.3% מהמתאכלסים הם מכוכב 0.

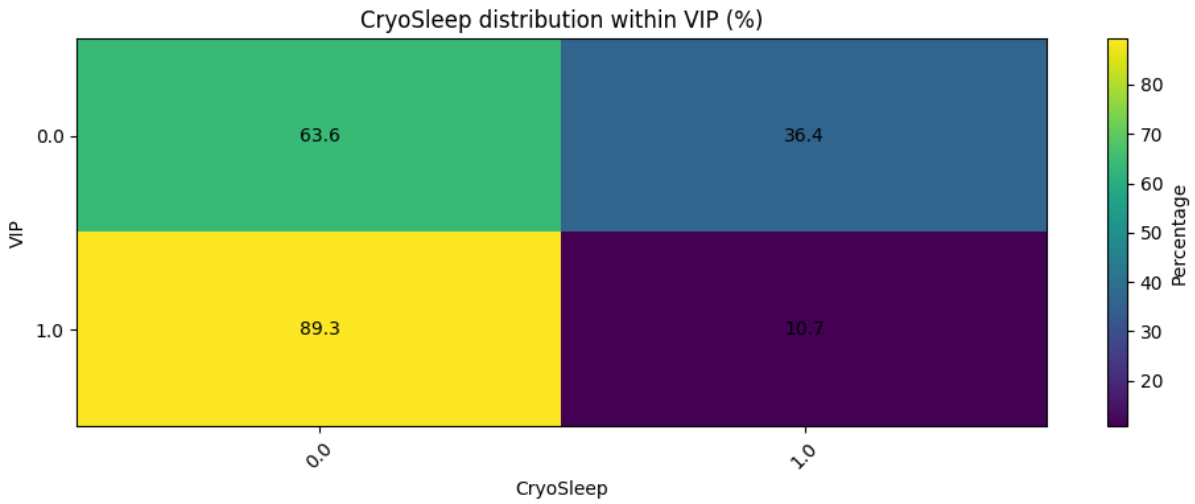
ובקומה 7 יש הסתברות של 80% לאנשים להיות מכוכב לכת 1, שהחלטנו שזה מספיק גבוה על מנת למלא את הנוסעים החסרים מקומה זאת.

- אחרי בדיקות אלו הגענו למצב שיש לנו רק 15 נוסעים ללא כוכב מוצא , שהחלטנו להוציא מהדאטה מאחר ולא מצאנו להם קשר העולה מהעין שלנו שיכול לתת לנו אינדיקציה מה הערך בהסתברות הגבוהה ביותר.

CryoSleep

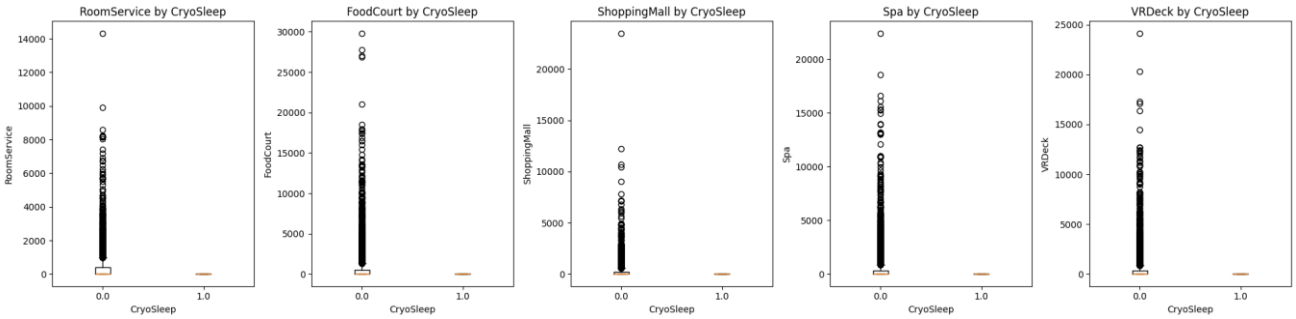
תחילה היו חסרים ל217 נוסעים ערך בעמודה.

- <u>קשר בין נוסעים שהם VIP לבין האם ישנו במהלך הטיסה:</u>



מאחר ו89.3% מתוך הנוסעים שלא ישנו, קנו את כרטיס הvip, החלטנו שזהו קשר מספיק חזק על מנת למלא את כל האנשים ללא Cryosleep ושהם vip כאנשים שהם לא ישנים במהלך הטיסה.

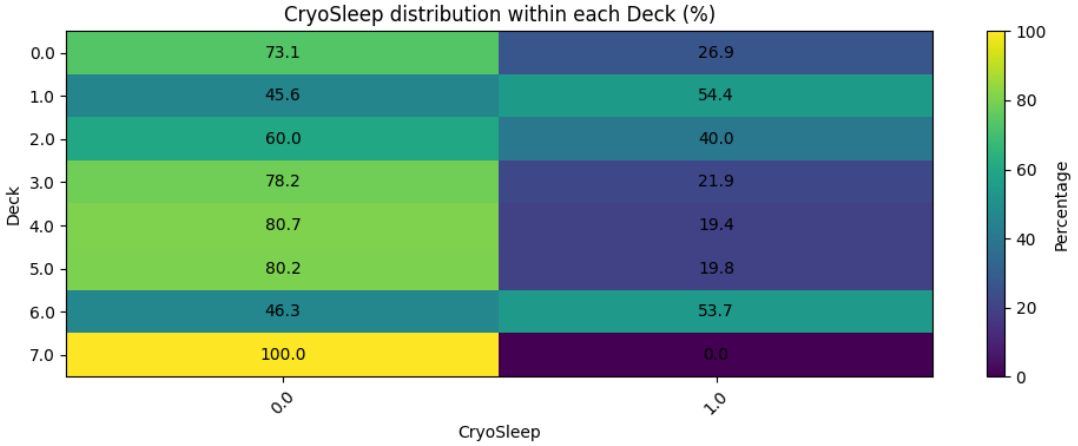
- <u>קשר בין שינת הנוסעים לבין הוצאותיהם במהלך הטיסה:</u>



ניתן על פי מידע זה, בסבירות של 100% , שכל האנשים שהחליטו שישנים במהלך הטיסה לא הוציאו שקל בשום עמודה בדאטה שלנו. ולכן כל הנוסעים שחסר להם עמודת השינה, והוציאו כסף במהלך הטיסה, החלטנו למלא להם שהם לא ישנו.

החלטנו לא למלא לאנשים שלא הוציאו שהם כן ישנו כרגע, ורצינו לבדוק קשרים אחרים לפני שנחליט החלטה כזאתי.

- <u>קשר הסתברותי בין מספר הקומה להחלטה אם ישנו במהלך הטיסה</u>



ראשית ניתן לראות בבירור שקומה מספר 7 , 100 אחוז מהאנשים שבקומה זאת ערים, ומילאנו את האנשים שחסר להם בקומה זאתי שהם ערים.

החלטנו גם בקומות 3,4,5 (בקומה 0 לא היו אנשים חסרים) למלא את האנשים שחסרה הערך בעמודה שהם ערים. החלטנו כי איזור ה80 אחוז זוהי הסתברות מספיק גבוהה על מנת לקבוע זאת.

- <u>סופי:</u>

לאחר בדיקות ומילויים אלו, הצלחנו למלא את כל המידע החסר ולא חסר לאף נוסע ערך CryoSleep.

Cabin

תחילה היו חסרים ל199 נוסעים ערך בCabin.

בדיקות:

- בדיקת קשר בין החדר שבו הנוסע נסע לבין מספר הקבוצה שהיה בה.

החלטנו לבודד כל קבוצה בנפשר לפי GroupId , ולבדוק האם הם מפוצלים למספר תאים או לתא בודד.

לאחר הבדיקה, גילינו כי לאחר הוצאת אנשים שאין להם עוד אנשים בGroupId , יש ל 87% מהקבוצות תא ייחודי, ולכן מילאנו את כל האנשים עם שמצאנו להם תא ייחודי בקבוצה גם את התא הזה.

unique_cabins	missing_cabins_count	
0	1	99
1	1	87
2	1	10
3	1	3

- סופי:

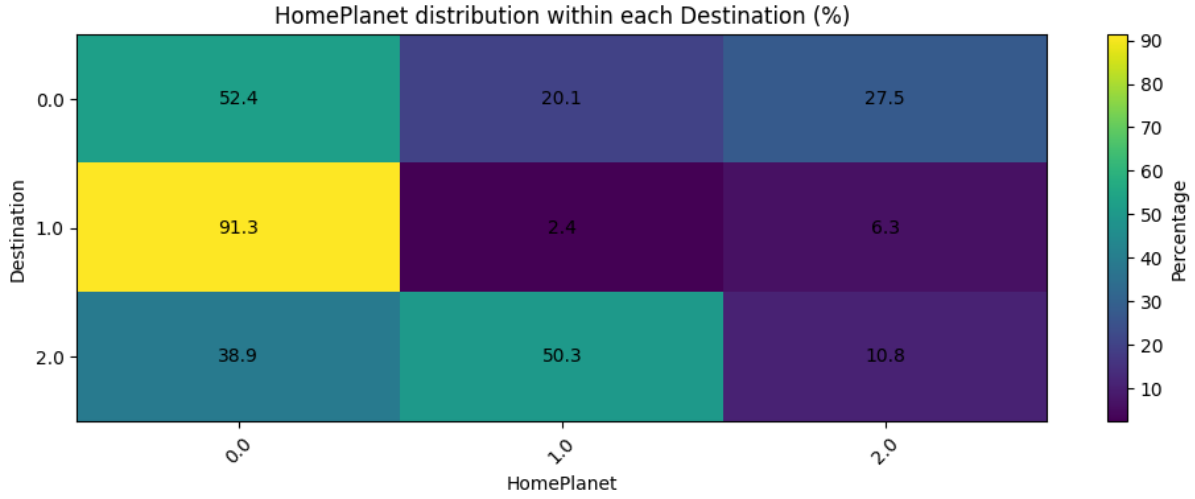
מאחר והתא מורכב גם מהקומה, גם ממספר החדר וגם מצד בספינה, ומאחר שלא מצאנו קשר ישיר בין החדר המדוייק לבין יתר התכונות, החלטנו להוריד את 110 הנוסעים בלי הערך Cabin. החלטנו שמאחר שאנחנו מפצלים את העמודה הזאתי ל3 עמודות שונות, אז ההיקף פגיעה במודל המסווג שלנו יכול להיות גדול מידי ולא נרצה לקחת את הסיכון הזה בגלל אחוז בודד מהנוסעים.

Destination:

תחילה היה חסר לנו בדאטה ל182 נוסעים ערך של יעד סופי.

בדיקות:

- בדיקת הסתברות מותנית של כוכב יעד בהסתמך על כוכב מוצא



אפשר לראות ש91.3% מהנוסעים שיצאו מכוכב לכת 0 הגיעו לכוכב לכת 1. ולכן לדעתנו הסתברות מספיק חזקה על מנת להשתמש במידע זה ולמלא את האנשים שחסר להם כוכב יעד והם מכוכב 1.

- מציאת קשר בין קבוצה שנוסעת ביחד ליעד משותף

Destination	Num_groups
1	5397
2	668
3	49

כפי שניתן לראות, לרוב הקבוצות שיש לנו יש יעד משותף יחיד, ולכן נמלא את הנוסעים שחסר להם יעד ובעלי קבוצה עם יעד ייחודי לאותו היעד

- מילוי יעדים לפי התא שהנוסע נמצא:

Destination	Num_cabins
0	103
1	5832
2	591
3	34

גם במקרה הזה, מעל 80 אחוז מהתאים נוסעים לאותו היעד, ולכן נמלא גם אותם לפי היעד של אותו התא.

- <u>סיום:</u>

לאחר סיום כל בדיקות אלו נשארנו עם 120 נוסעים שעדיין אין להם יעד, אך גם כאן כמו כוכב אם, החלטנו שאנחנו לא ממלאים את התאים ללא מידע רלוונטי שמבהיר לנו מאיפה הם ולאן הם נוסעים.

לכן החלטנו למחוק את נוסעים אלו מהדאטה שלנו על מנת לא להרוס את ביצועי המסווגים.

Age:

בטעינה הראשונית של המידע היה חסר לנו ב179 נוסעים הגיל שלהם.

לאחר בדיקות קשרים עם כל העמודות האחרות בדאטה, לא מצאנו מידע שימושי שיעזור לנו למלא את המידע הזה עבור הנוסעים.

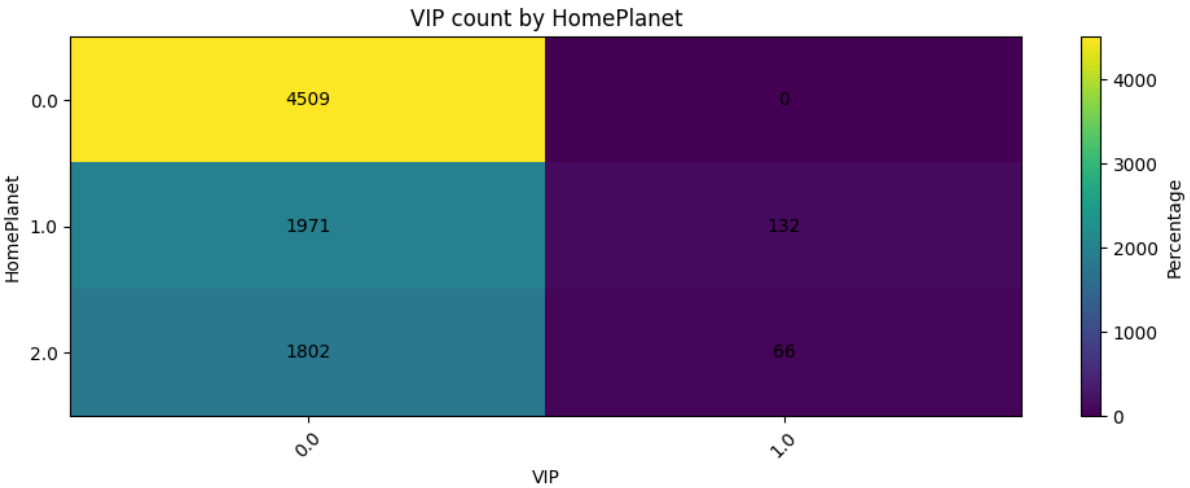
מאחר שראינו שיש טיפונת סטיית גילאים לכל קומה בסיפון, החלטנו לשים לכל נוסע את ממוצע הגילאים שקיימים לו באותה הקומה.

VIP:

תחילה היה חסר לנו 203 נוסעים שלא קיים להם האם הם וי איי פי או לא.

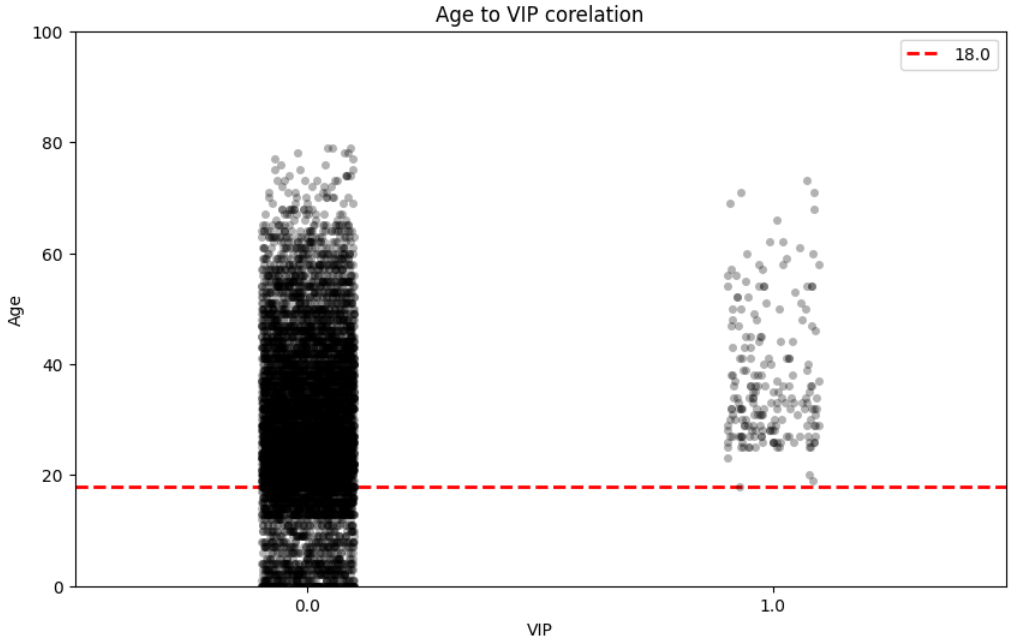
הבדיקות שמצאנו:

- הסתברות מותנית של וי איי פי לאחד שיודעים את כוכב המוצא:



ניתן לראות שכל הנוסעים שהגיעו מכוכב לכת 0, אינם VIP . לכם החלטנו למלא גם את הנוסעים החסרים כך.

- <u>בדיקת גיל בהקשר של וי איי פי</u>



אפשר לראות בגרף כאן שאין נוסעים שהם VIP מתחת לגיל 18, ולכן כל הנוסעים שחסר להם ומתחת לגיל 18 נשים להם 0.

- סופי:

אחרי בדיקות אלו, מאחר שאנחנו יודעים ש97 אחוז מהנוסעים הם לא VIP , נמלא את החסרים גם כלא .

Money:

החלטנו להתייחס לכל הכספים ביחד, וזאת מאחר שראינו כי ההתפלגות שלהם זהה.

כאן החלטנו פשוט שכל עמודה שחסר בה ערך, תקבל את הממוצע של יתר העמודות כספים של אותו הנוסע.

הורדת עמודות:

Name:

מאחר שאין לנו איך לייצג את המידע על השם במספרים, החלטנו לוותר על עמודה זאתי בתהליך הלמידה שלנו. השם לא נותן לנו ערך של ממש.

Cabin:

מאחר שפיצלנו את העמודה הזאתי כבר, החלטנו לא להשאיר גם את העמודה הזאתי.

One_hot Encoding

מאחר שיש לנו עמודות שמטרתן להיות קטגוריאליות ולא ערכים, נצטרך לפצלן לעמודות נפרדות על מנת לייצג אותם נכון בתהליך הלמידה שלנו של המסווגים (שלא תחשוב לדוגמא שכוכב לכת 2 גדול יותר פי 2 מכוכב לכת).

לכן החלטנו לממש את הקידוד הזה לכוכב האם ולכוכב היעד, וזאת על מנת לא לתת עליונות מספרים לכוכב ספציפי.

החלטנו לא לבצע קידוד על קומה, ומספר חדר מכיוון שלדעתנו יש סיכוי גבוה שגורמים אלו הם כן ייצוג של מיקום גיאוגרפי בספינה, ורצינו להשאירם ולבדוק זאת במסווגים.

הדאטה הסופי:

#	Column	Non-Null Count	Dtype
0	CryoSleep	8559 non-null	float64
1	Age	8559 non-null	float64
2	VIP	8559 non-null	float64
3	RoomService	8559 non-null	float64
4	FoodCourt	8559 non-null	float64
5	ShoppingMall	8559 non-null	float64
6	Spa	8559 non-null	float64
7	VRDeck	8559 non-null	float64
8	Transported	8559 non-null	int64
9	TotalSpend	8559 non-null	float64
10	GroupId	8559 non-null	int64
11	GroupMemberId	8559 non-null	int64
12	Deck	8559 non-null	float64
13	RoomNum	8559 non-null	float64
14	RoomSide	8559 non-null	float64
15	GroupSize	8559 non-null	int64
16	HomePlanet_0.0	8559 non-null	bool
17	HomePlanet_1.0	8559 non-null	bool
18	HomePlanet_2.0	8559 non-null	bool
19	Destination_0.0	8559 non-null	bool
20	Destination_1.0	8559 non-null	bool
21	Destination_2.0	8559 non-null	bool

כעת, לאחר שביצענו את הEDA ומילאנו את הדאטה במלואו על כל החלטותינו, נשתמש בו לבדיקת מסווגים שונים ונשווה בינהם.

יצירת המסווגים:

הקדמה:

על מנת שנשמור לאורך כל הדרך על אותו קו תכנוני הגדרנו כי נשתמש בK-fold cross-validation : כלומר החלטנו שאנחנו לפני שמתחילים לעבוד על המסווגים לפצל את הדאטה שלנו ל80 אחוז סדרת אימון, שבעזרתה נבצע קיפולי מידע ו"נבחן" את הדאטה על עצמו. ו20% ניקח לסדרת מבחן סופית, שתדמה לנו מצב אמיתי של מידע שנכנס למערכת. המידע הזה ישמש בסוף להשוואה בין כל המודלים שלימדנו במטרה לבחון אותם בשטח.

גם ב train_test_split וגם ב stratifiedKFold השתמשנו בrandom_state=42 על מנת לשמור על קו אחיד בין כל המודלים ולעבוד בדיוק על אותו הדאטה (במקום לעבוד במחברת אחת ארוכה).

לאורך כל בניית המודלים אני לא אתייחס לנתוני המבחן שלנו, וגם לא לנתוני הקיפול, וזאת מאחר ואנחנו נבצע בסוף קובץ נפרד שיריץ כל מודל הכי טוב שיצא לנו בתורו עם ההייפר פרמטרים הכי טובים שמצאנו מסדרת האימון, ונדמה מצב אמיתי שבו קיבלנו דאטה חדש וננסה לסווג מידע בזמן אמת.

תהליך בניית המודלים:

1. טעינת הנתונים למחברת ופיצול ל features-x ול labels-y.
2. פיצול כל הדאטה ל20 אחוז סדרת מבחן ו80 אחוז סדרת אימון.
3. יצירת cross_validation שמחלק את סדרת האימון ל5 חלקים שווים וכל פעם משתמש באחד מהם כסדרת מבחן פנימית.
4. יצירת קומבינציות של כל ההייפר פרמטרים על מנת להגיע למודל בעל אחוז הדיוק הגבוהה ביותר (מאחר ואנחנו ביצענו KFold של 5 אז כל קומבינציה של ההייפר פרמטרים מביאה לנו 5 אחוזי דיוק ואנחנו נעבוד על הממוצע שלו מאחר ובמודל הסופי נבנה אותו על כל הדאטה בלי קיפולים).
5. בניית המודל בעל הפרמטרים האופטימלים.

Standard_Scalar()

StandardScaler הוא תהליך נרמול נתונים שבו כל משתנה (feature) מותאם כך שהממוצע שלו יהיה 0 וסטיית התקן שלו תהיה 1. מטרת הפעולה היא להביא את כל המשתנים לאותה סקאלה מספרית, מאחר שבנתונים אמיתיים יש משתנים בעלי טווחים שונים מאוד (למשל במקרה שלנו גיל לעומת נתוני הוצאות כספיות). ללא נרמול, משתנים בעלי ערכים גדולים עלולים להשפיע יותר על תהליך הלמידה, גם אם אינם חשובים יותר מבחינה מידעית.

השתמשנו בזה על מנת לשפר את ביצועי המודלים ולא לתת לעמודה אחת "כוח" גדול מידי רק על סמך הטווח המספרי שלו.

Pipeline():

הוא מנגנון המאפשר לאחד מספר שלבי עיבוד ולמידה לזרימה אחת סדורה. במסגרת ה-Pipeline כל שלב מבוצע לפי סדר קבוע – לדוגמה עיבוד מקדים של הנתונים (כמו נרמול באמצעות StandardScaler) ולאחריו אימון המודל עצמו. שימוש ב-Pipeline מבטיח שכל פעולות העיבוד המקדימות נלמדות אך ורק על סט האימון ומיושמות בצורה זהה גם על סט הבדיקה, ובכך מונע זליגת מידע (Data Leakage) מתהליך האימון לתהליך ההערכה. בנוסף, ה-Pipeline משפר את הקריאות והסדר של הקוד, מפשט את תהליך האימון והבדיקה, ומאפשר לבצע השוואה הוגנת ועקבית בין מודלים שונים תחת אותה תצורת עיבוד נתונים.

GridSearchCV:

הוא מנגנון לחיפוש שיטתי של היפר־פרמטרים במודלי למידת מכונה. במסגרת תהליך זה נבדקות באופן אוטומטי מספר קומבינציות של ערכי פרמטרים שהוגדרו מראש, כאשר כל קומבינציה מוערכת באמצעות ולידציה צולבת (Cross-Validation) על סט האימון בלבד. מטרת השיטה היא לבחור את ערכי ההיפר־פרמטרים שמובילים לביצועים הטובים ביותר, מבלי להסתמך על חלוקה בודדת של הנתונים. שימוש ב-GridSearchCV מאפשר לנו לקבל מודל יציב ואמין יותר, להפחית תלות במקריות של חלוקת הנתונים, ולשפר את יכולת ההכללה של המודל על נתונים חדשים.

(את יישום זה לשם הלימוד השתמשנו רק בKNN , את יתר המסווגים בדקנו את ההייפר פרמטרים ידנית ללא GridSearchCV.

בניית המודלים:

KNN:

הסבר:

KNN הוא אלגוריתם סיווג המבוסס על עיקרון הדימיון בין דוגמאות. מודל זה אינו מבצע תהליך למידה מפורש שבו נבנים פרמטרים פנימיים או פונקציות החלטה כלליות, ובמקום זאת שומר את דוגמאות האימון ומשתמש בהם בזמן החיזוי.

כאשר הוא מקבל דוגמא חדשה לחיזוי האלגוריתם מחשב את המרחקים בין הדוגמא החדשה לבין הדוגמאות אימון שיש לנו, ולאחר חישוב המרחקים נבחר את K הדוגמאות הקרובות ביותר מבחינת שכנות והמחלקה תיקבע על פיהם.

האלגוריתם מניח כי נקודות הקרובות זו לזו נוטות להשתייך לאותה המחלקה וגבול ההחלטה נקבע בצורה מקומית.

הייפר פרמטרים שבחנו במודל:

- מספר השכנים K: קובע כמה דוגמאות קרובות ישתתפו בהחלטת הסיווג של כל דאטה חדש. (ערך קטן יכול להוביל למודל רגיש מידי לרעש, ואילו ערך גדול מידי יוצר גבול החלטה פחות רגיש אבל יכול לגרום לאיבוד מידע באיזורי מעבר).

- שיטת שקלול השכנים (weights):

רמת ההשפעה של דוגמאות קרובות לעומת רחוקות בתחום

- **Uniform** - כל שכן משפיע באותה מידה.
- **Distance** - שכנים קרובים יותר מקבלים משקל גבוה יותר.

- מדד המרחק (metric) :

מגדיר כיצד מחושב המרחק בין הדוגמאות

- Euclidean distance- מרחק גיאומטרי רגיל.
- Manhattan distance- סכום ההפרשים המוחלטים בין התכונות.

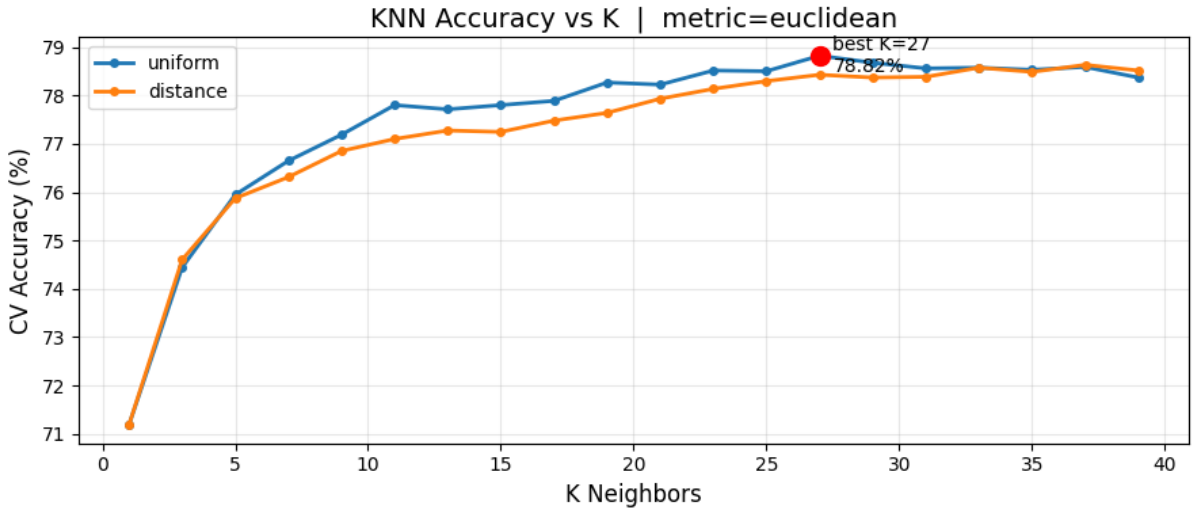
תוצאות המודל:

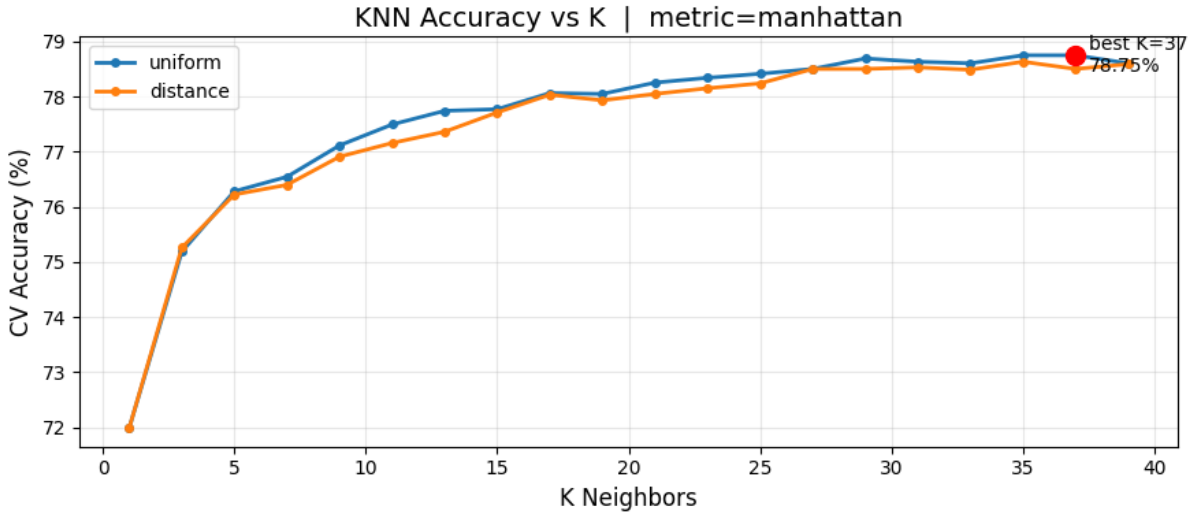
	param_knn__n_neighbors	param_knn__weights	param_knn__metric	mean_test_score	std_test_score	rank_test_score
26	27	uniform	euclidean	0.788231	0.013537	1
76	37	uniform	manhattan	0.787500	0.013157	2
74	35	uniform	manhattan	0.787500	0.013362	3
68	29	uniform	manhattan	0.786916	0.013398	4
28	29	uniform	euclidean	0.786770	0.012731	5

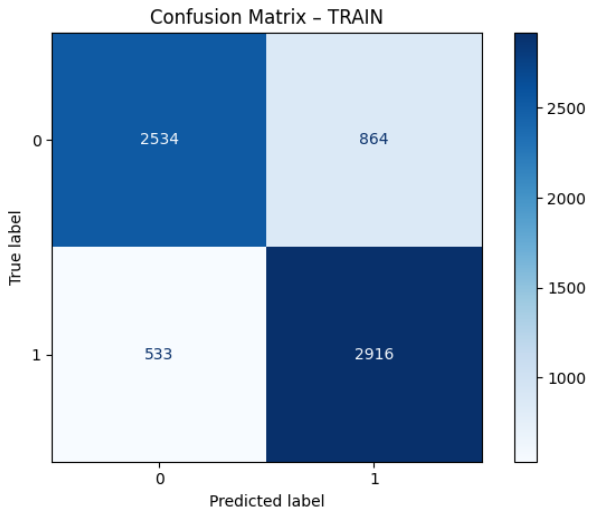
כפי שניתן לראות בתמונה, ההייפר פרמטרים שנתנו את ממוצע אחוז הדיוק הגבוה ביותר הם:

- מספר השכנים K: 27.
- שיטת שקלול השכנים: uniform.
- מדד המרחק: euclidean.

המודל נתן ממוצע של 78.82% דיוק.







ניתן לראות במטריצת הערבול כאן סטייה קלה בזיהוי שגוי של לייבל אמת 0 עומת 1.

QDA:

הסבר:

QDA הוא אלגוריתם סיווג סטטיסטי המבוסס על הסתברויות והפלגויות של הנתונים.

המודל מניח כי כל מחלקה בדאטה מתפלגת לפי התפלגות נורמלית, ולכל מחלקה קיימת מטריצת שונות שונה.

כאשר מתקבלת דוגמא חדשה לחיזוי, האלגוריתם מחשב את ההסתברות של הדוגמא עבור כל אחד מהמחלקות לפי ההתפלגות שנלמדה למחלקה, והדוגמא תסתווג להסתברות הגבוהה ביותר.

הייפר פרמטרים שבחנו במודל:

למרות שבכיתה לא למדנו על הייפר פרמטר במודל זה, לאחר בדיקה מצאנו שיש אפשרות לשנות את reg_param שתפקידו לייצב את המודל בזמן חישוב השונות בין התכונות בכל מחלקה. כאשר הערך שלו נמוך, המודל מתאים את עצמו בצורה חזקה יותר לנתונים אך עלול להיות רגיש לרעש ולשינויים קטנים בדאטה. כאשר הערך גבוה יותר, המודל נהיה יציב ופשוט יותר אך עשוי לאבד חלק מהמידע. לכן בוצעה בדיקה של מספר ערכים שונים של הפרמטר כדי למצוא את האיזון בין דיוק לבין יציבות המודל.

תוצאות בניית המודל:

```
==================================
BEST QDA - TRAIN SET PERFORMANCE
==================================
Accuracy : 74.24%
==================================
```

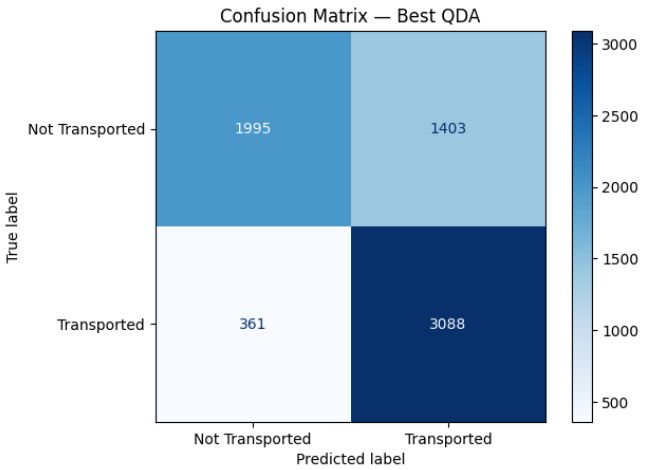
	rank_test_score	mean_test_score	std_test_score	param_qda__reg_param
0	1	75.799715	1.613228	0.000000
1	2	73.448273	1.640098	0.200000
2	3	73.243787	1.603521	0.189474
3	4	73.024692	1.673571	0.178947
4	5	72.966276	1.674649	0.168421

ניתן לראות כי הערך הטוב ביותר עבור reg_param היה 0, כלומר ללא regularization נוסף. כבל שערך הפרמטר עלה, חלה ירידה בביצועים, דבר המצביע על כך שהמודל הצליח ללמוד את מבנה הנתונים בצורה טובה גם ללא החלקה נוספת של מטריצת השונות.

הייפר פרמטר זה גרם לירידה בביצועי המודל ולכן אין בו צורך.

השיפור המשמעותי שהיה לנו במודל זה היה שימוש ב StandardScaler() (71% לפני השימוש ו74.35% אחרי).

בגלל ש QDA-מבוסס על חישוב שונות ופיזור של הנתונים, ההבדלים בסקאלה גרמו לו ללמוד גבולות החלטה פחות טובים. אחרי הנרמול כל הפיצ'רים משפיעים בצורה דומה, ולכן המודל מצליח להבין טוב יותר את המבנה של הנתונים וזה הוביל לשיפור משמעותי בדיוק המסווג.



לפי מטריצת הערבות ניתן לראות בתוצאות שיש זיהוי שגוי הרבה יותר כאשר הנוסע לא עבר טרנספורמציה, לעומת המקרים שבהם הוא כן עבר.

יש נטייה למודל שנדבר עליה בחלק של המבחן.

LDA

הסבר:

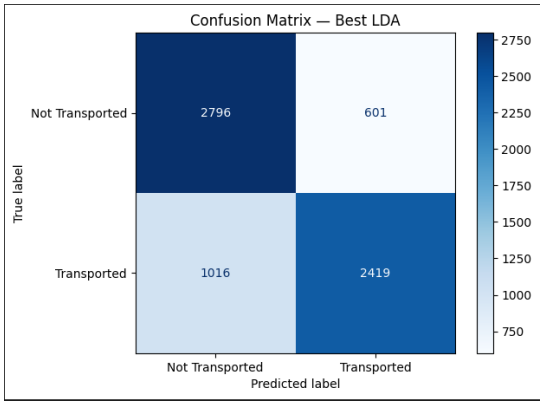
LDA הוא אלגוריתם סיווג שמטרתו להפריד בין מחלקות שונות על ידי מציאת גבול החלטה לינארי ביניהן. המודל מניח שכל מחלקה מתפלגת בצורה נורמלית ושכל המחלקות חולקות את אותה מטריצת שונות (בשונה מQDA ששם לכל מחלקה יש מטריצת שונות משלה), אך לכל מחלקה יש ממוצע שונה.

על בסיס ההנחות האלה LDA ,מחשב את הכיוון במרחב הנתונים שבו ההפרדה בין המחלקות היא הטובה ביותר, ובונה גבול החלטה לינארי שמאפשר לסווג דוגמאות חדשות.

תוצאות בניית המודל:

```
==========================
LDA — TRAIN PERFORMANCE
==========================
Accuracy : 76.63%
==========================
```

גם כאן השימוש ב standard_scalar שיפר את המודל (76.63% דיוק במקום 72%) .



אפשר לראות לפי המטריצה כאן שפה דווקא היה למסווג יותר קשה לסווג אנשים שכן עבר טרנספורמציה, לעומת הQDA שהיה הפוך, אבל כאן גם יש יותר יציבות בין שגיאות הסיווג והנטייה של המסווג יותר קטנה (נוכל לתת נתונים מדוייקים רק אחרי סדרת המבחן).

GNB

הסבר:

GNB הוא אלגוריתם סיווג שמבוסס על משפט בייס. המודל מניח שכל פיצ'ר מתפלג בצורה נורמלית (Gaussian), ושכל הפיצ'רים בלתי תלויים אחד בשני בתוך כל מחלקה. לפי ההנחות האלה, הוא מחשב את ההסתברות שכל דוגמה שייכת לכל מחלקה, ובוחר את המחלקה בעלת ההסתברות הגבוהה ביותר.

הייפר פרמטרים שבחנו במודל:

אין הייפר פרמטרים, אלגוריתם שמסתמך על התפלגות גאוסית.

תוצאות בניית המודל:

```
==================================
BEST GNB – TRAIN SET PERFORMANCE
==================================
Accuracy : 72.31%
==================================
```

אפשר לראות כאן שאחוז הדיוק של המסווג הזה הרבה יותר נמוך, וגם לאחר שימוש בקוונטיזציה לטווחים לא הצלחנו לשפר את אחוז הדיוק של המסווג.

אנחנו יכולים להניח בגלל שהמודל הזה אומר שכל התכונות בלתי תלויות אחת לשנייה, ומאחר שיש לי תכונות קטגוריאליות שיצרנו עבורן one_hot , אז החיזוי היה לוקה בחסר.

Decision Tree

הסבר:

Decision Tree הוא אלגוריתם סיווג המבוסס על מבנה של עץ החלטות, שבו המודל מחלק את הנתונים באופן הדרגתי לפי תנאים על הפיצ'רים השונים. בכל צומת בעץ נבחר הפיצ'ר שמאפשר את ההפרדה הטובה ביותר בין המחלקות, עד להגעה לעלים שבהם מתקבלת החלטת הסיווג הסופית.

הייפר פרמטרים שבחנו במודל:

- max_depth: קובע את העומק המקסימלי של העץ
- min_samples_leaf: מספר הדוגמאות המינימלי בכל עלה
- min_samples_split: קובע כמה דוגמאות מינימום צריך כדי לבצע פיצול של צומת
- criterion : קובע איך המודל מחליט מה הפיצול הכי טוב בכל צומת בעץ
  - Gini
  - Entropy

תוצאות בניית המודל:

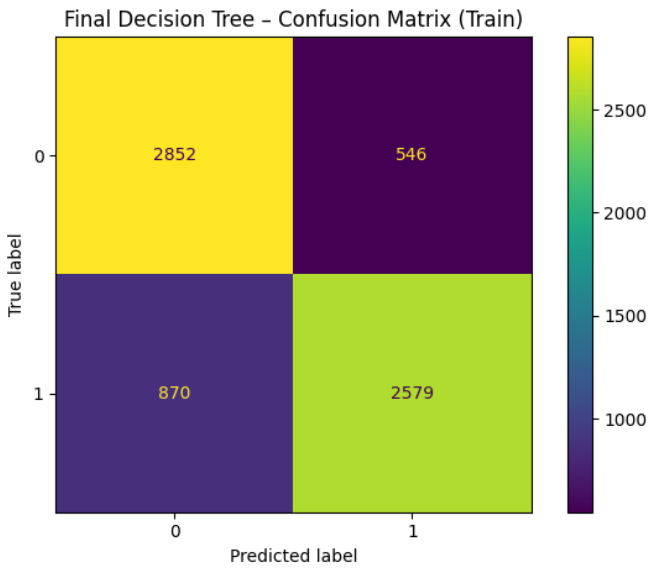
	Parameter	Value
0	criterion	entropy
1	max_depth	7
2	min_samples_leaf	87
3	min_samples_split	88
4	max_leaf_nodes	38

אלו ההייפר פרמטרים שנקבעו אחרי כוונון העץ לסיווג אופטימלי האפשרי.

עומק עץ מוגבל ומספר מינימלי גבוה יחסית של דוגמאות בכל עלה ובכל פיצול מסייעים במניעת Overfitting ומשפרים את היציבות של המודל על נתונים חדשים. בנוסף, הגבלה על מספר העלים מאפשרת שמירה על מבנה עץ פשוט וברור יותר, תוך שמירה על ביצועי סיווג טובים.

```
==========================================
BEST Decision Tree – TRAIN SET PERFORMANCE
==========================================
Accuracy : 79.32%
------------------------------------------
```

אחרי כיוונון ההייפר פרמטרים המודל נתן אחוז דיוק לא גבוה יחסית של סדרת האימון.



העץ יחסית מאוזן ובלי סטיות לצד אחד.

Random Forest

הסבר:

Random Forestהוא אלגוריתם סיווג המבוסס על אוסף של עצי החלטה (Decision Trees) הפועלים יחד כקבוצה. במקום להסתמך על עץ יחיד, המודל מאמן מספר רב של עצים על תתי־דגימות שונות מהנתונים ובכל פיצול משתמש בתת־קבוצה אקראית של פיצ'רים. במהלך החיזוי, כל עץ מצביע למחלקה מסוימת וההחלטה הסופית מתקבלת לפי רוב קולות. גישה זו מפחיתה את הסיכון ל־ Overfittingומשפרת את היציבות והדיוק בהשוואה לעץ החלטה יחיד.

הייפר פרמטרים שבחנו במודל:

- n_estimators: מספר העצים ביער
- max_depth: העומק המקסימלי של כל עץ
- Max_features: מספר התכונות הזמינות לעץ בכל פיצול.
- Min_sanple_leaf:מספר הדוגמאות המינימלי הנדרש לכל עלה בעץ.

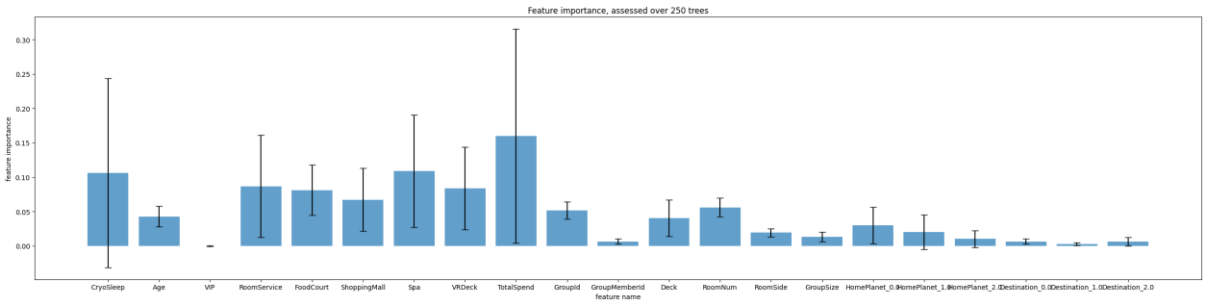
תוצאות בניית המודל:

```
========================================
BEST Random Forest — TRAIN SET PERFORMANCE
========================================
Accuracy : 87.62%
========================================
```

הגענו לאחוז דיוק גבוה יחסית על סדרת האימון.

	Hyperparameter	Best Value
0	n_estimators	250
1	max_depth	21
2	max_features	sqrt
3	min_samples_leaf	7

יצא לנו 250 עצים ביער, עם עומק מקסימלי של 21 החלטות ומינימום דוגמאות בכל עלה של 7 דוגמאות.



ניתן לראות מגרף החשיבות של כל תכונה שיש חשיבות יחסית גבוהה לכמות הבזבוזים של הנוסע על האם הוא נעלם, וגם על האם הוא היה בשינה במהלך הנסיעה. (בEDA ראינו שיש קשר ישיר בין שינה להוצאות אז יכול להיות שבגלל זה הם ביחד).

SVM

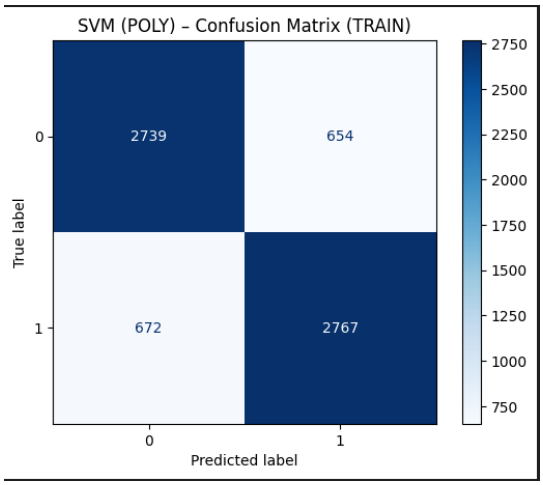
הסבר:
SVM הוא אלגוריתם סיווג שמטרתו למצוא את הגבול האופטימלי שמפריד בין המחלקות במרחב הנתונים. המודל בוחר גבול החלטה שממקסם את המרחק (margin) בין הדוגמאות הקרובות ביותר מכל מחלקה.

הייפר פרמטרים שבחנו במודל:

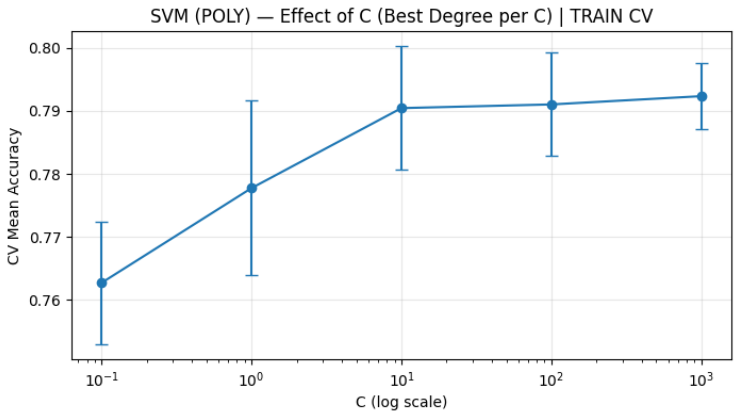
- C: קובע את האיזון בין התאמה טובה לנתוני האימון לבין הכללה.
- Degree: המעלה של הפולינום כאשר אנחנו משתמשים ב poly kernel.

תוצאות בניית המודל:

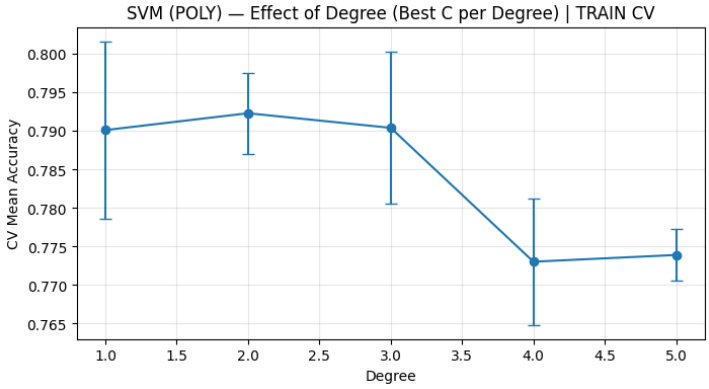
```
==========================================
BEST SVM (POLY) — TRAIN SET PERFORMANCE
==========================================
Accuracy : 80.96%
==========================================
```



המודל נתן 80.96% דיוק נראה שאינו נוטה לצד אחד בשגיאות שלו.



הגרף מציג את השפעת הפרמטר **C** על ביצועי מודל SVM כאשר עבור כל ערך של C נבחר ערך ה degree-האופטימלי.



הגרף מציג את השפעת הפרמטר **degree** במודל SVM כאשר לכל ערך של degree נבחר ערך ה־ Cהאופטימלי.

Logistic Regression:

הסבר:

Logistic Regression הוא אלגוריתם סיווג לינארי שמחשב את ההסתברות שכל דוגמה שייכת למחלקה מסוימת. המודל משתמש בשילוב לינארי של התכונות ולאחר מכן מעביר את התוצאה דרך פונקציית Sigmoid, שממירה את הערך להסתברות בין 0 ל-1. על בסיס ההסתברות הזו מתקבלת החלטת הסיווג הסופית.

הייפר פרמטרים שבחנו במודל:

- C: פרמטר הרגולריזציה הראשי.
- Penalty: סוג הרגולריזציה.
  - L2: ברירת מחדל, מקטין משקולות גדולות ושומר על יציבות.
  - L1: רגולריזציה שמקטינה את משקלי המודל ועלולה לאפס חלק מהם לגמרי.

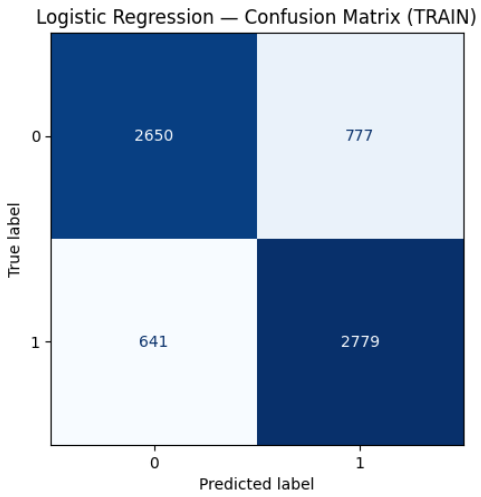
תוצאות בניית המודל:

	penalty	C	cv_mean	cv_std
9	l1	100.00	0.7907	0.0114
8	l2	100.00	0.7907	0.0114
7	l1	10.00	0.7907	0.0114
6	l2	10.00	0.7906	0.0112

```
==========================================
BEST Logistic Regression – TRAIN SET PERFORMANCE
==========================================
Accuracy : 79.29%
==========================================
```

אחרי כיוונון ההייפר פרמטרים של המודל יצא כי המודל החזק ביותר הוא L1עם c=100.

ואחוז הדיוק של מודל זה הוא 79.07%.



גם כאן אין נטייה של המודל לאיזשהו צד.

MLP

הסבר:

הוא מודל רשת נוירונים מלאכותית המורכב ממספר שכבות של נוירונים: שכבת קלט, שכבות חבויות ושכבת פלט. כל נוירון מקבל קלטים, מבצע חישוב מתמטי ומעביר את התוצאה הלאה באמצעות פונקציית הפעלה לא לינארית. בזכות השכבות והלא־לינאריות, המודל מסוגל ללמוד קשרים מורכבים ולא לינאריים בין התכונות ולבצע סיווג מדויק יותר בדאטה מורכב.

הייפר פרמטרים שבחנו במודל:

- hidden_layer_sizes: מספר הנוירונים בכל שכבה חבויה ומבנה הרשת.
- Activation: פונקציית ההפעלה של הנוירונים
  - RELU
  - TANH
- Alpha: פרמטר רגולריזציה.

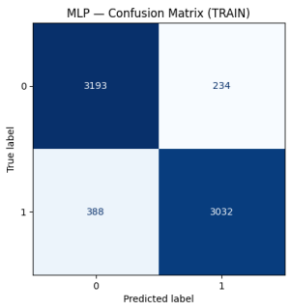
תוצאות בניית המודל:

```
=====================================
BEST MLP — TRAIN SET PERFORMANCE
=====================================
Accuracy : 91.05%
=====================================
```

	Parameter	Value
0	hidden_layer_sizes	(128, 64)
1	alpha	0.01
2	activation	tanh

לאחר כיוונון ההייפר פרמטרים מצאנו כי יש לנו 2 שכבות חבויות, שבראשונה יש 128 נוירונים ובשנייה 64, עם alpha של 0.01 ופונקציית אקטיבציה tanh.

ונותן אחוז דיוק של 91.05%. דיוק גבוהה במיוחד שיכול להצביע על overfitting, נגלה בסדרת המבחן בהמשך.



Xgboost:

הסבר:

Xgboost הוא אלגוריתם סיווג מתקדם המבוסס על שיטת Boosting, שבה נבנית סדרה של עצי החלטה בצורה הדרגתית כך שכל עץ חדש מתקן את הטעויות של העצים הקודמים. בניגוד ל־Random Forest שבו העצים פועלים במקביל, ב־XGBoost העצים נבנים בצורה רציפה ולכן המודל מסוגל ללמוד דפוסים מורכבים מאוד בנתונים.

הייפר פרמטרים שבחנו במודל:

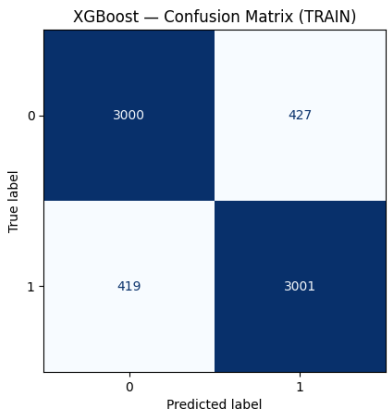
- **max_depth:** העומק המקסימלי של כל עץ.
- **n_estimators:** מספר העצים במודל.
- **learning_rate:** קצב הלמידה של המודל.

תוצאות בניית המודל:

```
=====================================
BEST XGB — TRAIN SET PERFORMANCE
=====================================
Accuracy : 87.64%
=====================================
```

	Parameter	Value
0	max_depth	5.0
1	n_estimators	150.0
2	learning_rate	0.1

לאחר כיוונון הפרמטרים מצאתי כי העומק המקסימלי של כל עץ הוא 5, שיהיו 150 עצים ושקצב הלמידה יהיה 0.1.



השוואת המודלים על סדרת המבחן:

Model Comparison (Train/Test Accuracy)		
Model	**Train Accuracy**	**Test Accuracy**
XGBoost	87.64%	80.96%
Random Forest	87.62%	80.02%
Logistic Regression	79.29%	79.32%
SVM	80.96%	79.26%
Decision Tree	79.68%	77.80%
KNN	79.16%	77.63%
MLP	97.59%	77.04%
LDA	76.92%	76.64%
QDA	76.57%	76.52%
GNB	72.08%	72.49%

ניתן לראות שהמודל הכי טוב שיצא לנו ביחס לסדרת המבחן הוא המודל XGBoost.

הסבר על XGBoost ולמה לדעתנו נבחר:

לאחר השוואה בין המסווגים השונים, נבחר מודל XGBoost כמסווג הסופי שניקח להמשך, מאחר והוא מציג את ביצועי הסיווג הגבוהים והיציבים ביותר על נתונים שלא נראו במהלך האימון (גם בתהליך הקרוס וולידיישן הגיע למעל 80 אחוז דיוק בממוצע).

תהליך הבחירה התבסס על השוואה שיטתית של דיוק על קבוצות האימון והבדיקה, וכן על ניתוח גרפי של ביצועי המודלים השונים.

הכללה

אפשר לראות בטבלת ההשוואות בעמוד קודם, שישנו הפרש של פחות מ7% בין סדרת המבחן לסדרת האימון, מה שמעיד לנו שלא נוצר כאן תהליך של Overfitting בתהליך הלמידה, פער זה מצביע על יכולת הכללה טובה. (בשונה מmlp, אשר הביא ביצועים מאוד גבוהים על סדרת האימון, אבל בסדרת במבחן הראה שהוא מותאם מידי לדאטה של האימון ולא יכול לסווג כהלכה דאטה חדש).

ערכי ההייפק פרמטרים שנבחרו:

הערכים שנבחרו במסווג זה נבחרו על ידי Cross-validation. ואלו שנמצאו אופטימליים הם:

- n_estimators:150
- Max_depth:5
- Learning_rate:0.1

ניתוח מדדי הסיווג:

	precision	recall	f1-score	support
0	79.20%	81.71%	80.43%	82000.00%
1	82.68%	80.27%	81.46%	89200.00%
accuracy	80.96%	80.96%	80.96%	80.96%

לאחר ניתוח המדדים, המודל מראה כי המסווג השיג ביצועים יחסית מאוזנים בין 2 המחלקות שלנו.

עבור מחלקה 0:

79.2% מהפעמים שאמר שהמחלקה היא 0, הוא צדק.

81.7% מהדגימות שהיו באמת 0 הוא הצליח למצוא.

בסך הכל דיוק ממוצא של 80.43%

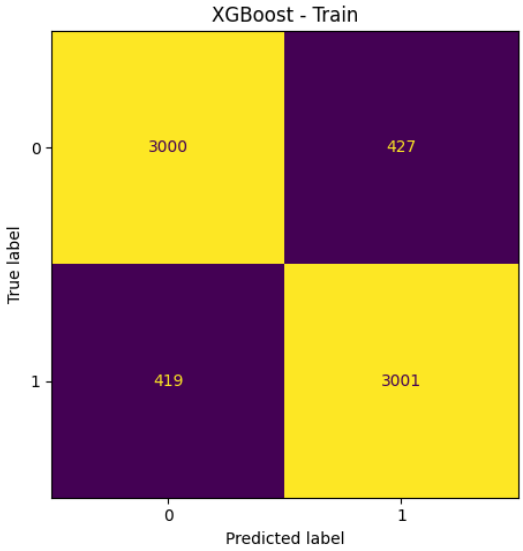
עבור מחלקה 1:

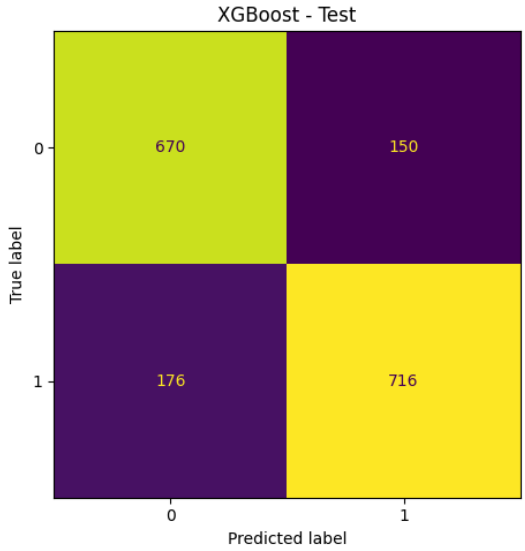
82.68% מהפעמים שאמר שהמחלקה היא 0, הוא צדק.

80.27% מהדגימות שהיו באמת 0 הוא הצליח למצוא.

בסך הכל דיוק ממוצא של 81.46%

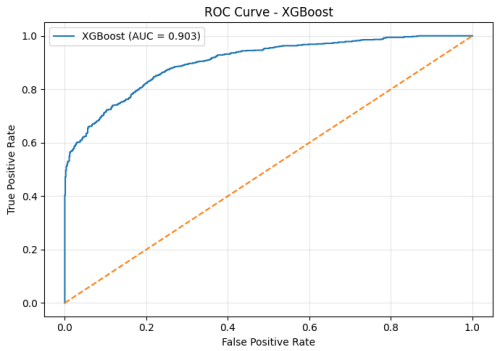
ישנה סטייה ממש קטנה לכיוון זיהוי 1 (שהנוסע שוגר), אבל מאוד זניחה ואפשר להגיד שהמודל נחשב יחסית מאוזן.

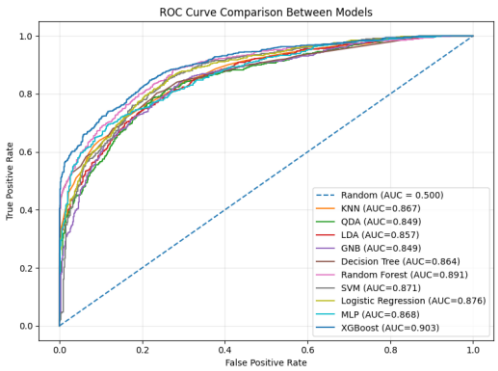




מטריצות הערבול מאשרות שהמודל יחסית מאוזן , בין עם הקשר לסדרת האימון והמבחן, ובין אחוז הדיוק בין מחלקה 0 למחלקה 1.

ניתוח גרף ROC:

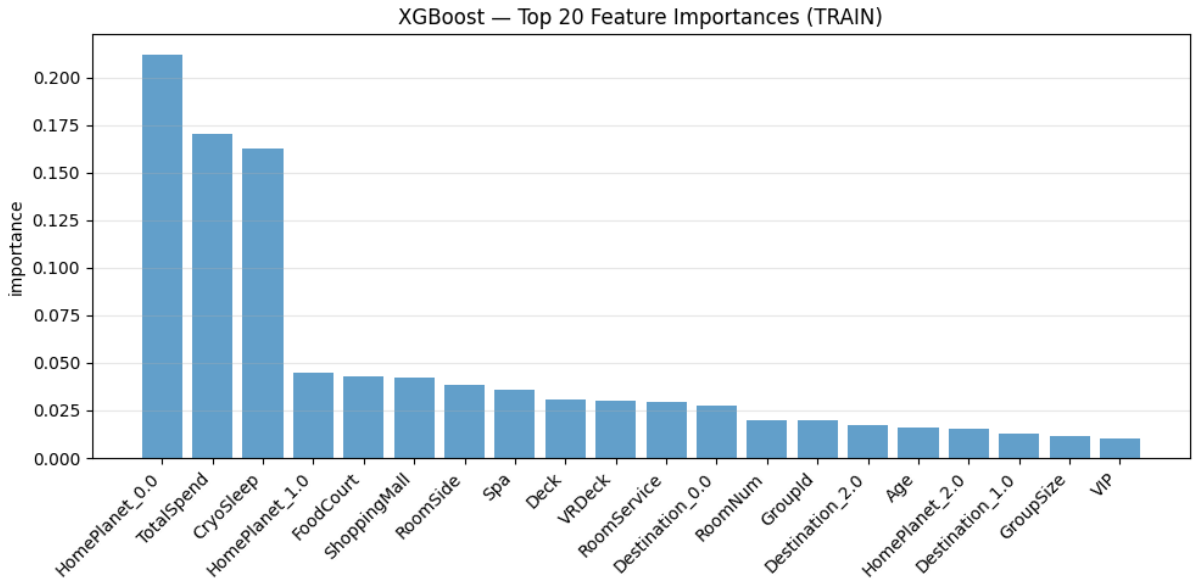




אפשר לראות שעבור תוצאות המסווג על סדרת המבחן, xgboost מציג את הביצועים הגבוהים ביותר עם 0.903 שזהו יכולת הבחנה הגבוהה ביותר מבין המודלים שקיבלנו.

(הפערים בין עקומות ה־ROC אינם גדולים מאוד, דבר המעיד כי קיימת תקרת ביצועים מסוימת לבעיה זו, אולם XGBoost מצליח להפיק את הביצועים הגבוהים ביותר בתוך מגבלה זו.)

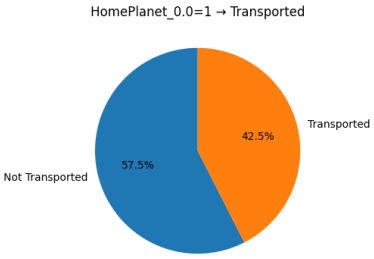
חשיבות התכונות:

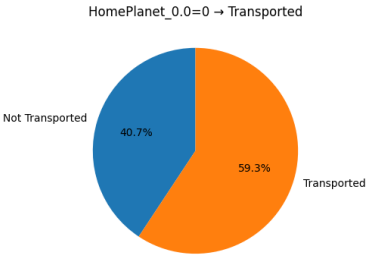


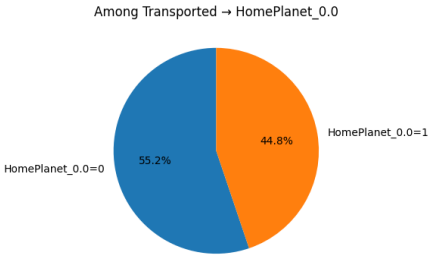
הגרף הזה שמציג את חשיבות התכונות של המסווג מציג לנו את התרומה היחסית של כל משתנה למודל.

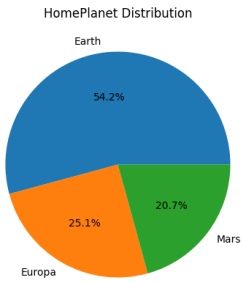
ניתן לראות כי יש לנו 3 תכונות שבלטו מעל כולם בחשיבותם:

HomePlanet_0: עמודה שקובעת אם הנוסע הגיע מכדור הארץ (לאחר קידוד HomePlanet ל3 עמודות נפרדות, דבר שעזר לנו להגיע למסקנה שתוצאות כוכב לכת בודד הרבה יותר חשובות מהאחרים.



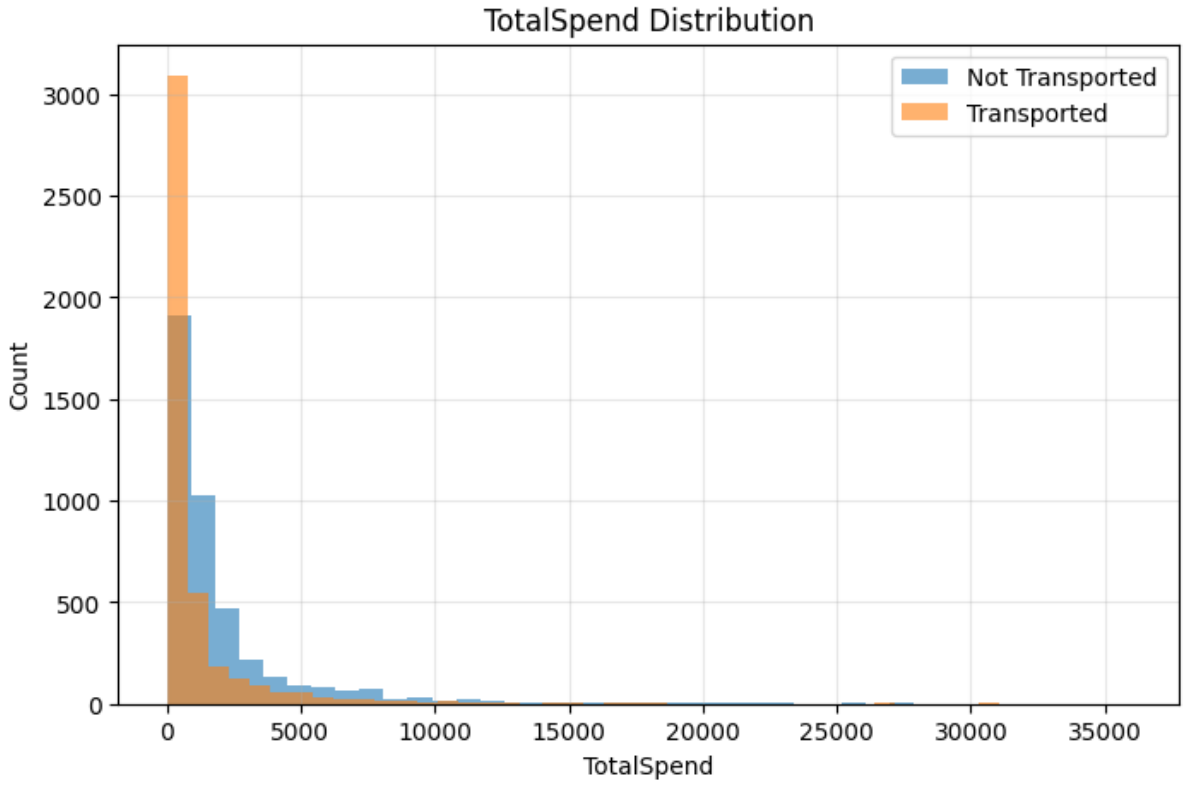






אפשר לראות שיש נטייה קטנה- יש סיכוי יותר גדול לאנשים שהגיעו מכדור הארץ לא לעבור טרנספורמציה. מאחר שרוב האנשים הגיעו מכדור הארץ, ומאחר וזה מודל שמבוסס על עצים ויש לו השפעה בעזרת עמודות אחרות אז זה הגיוני שנראה רק סטייה קטנה.

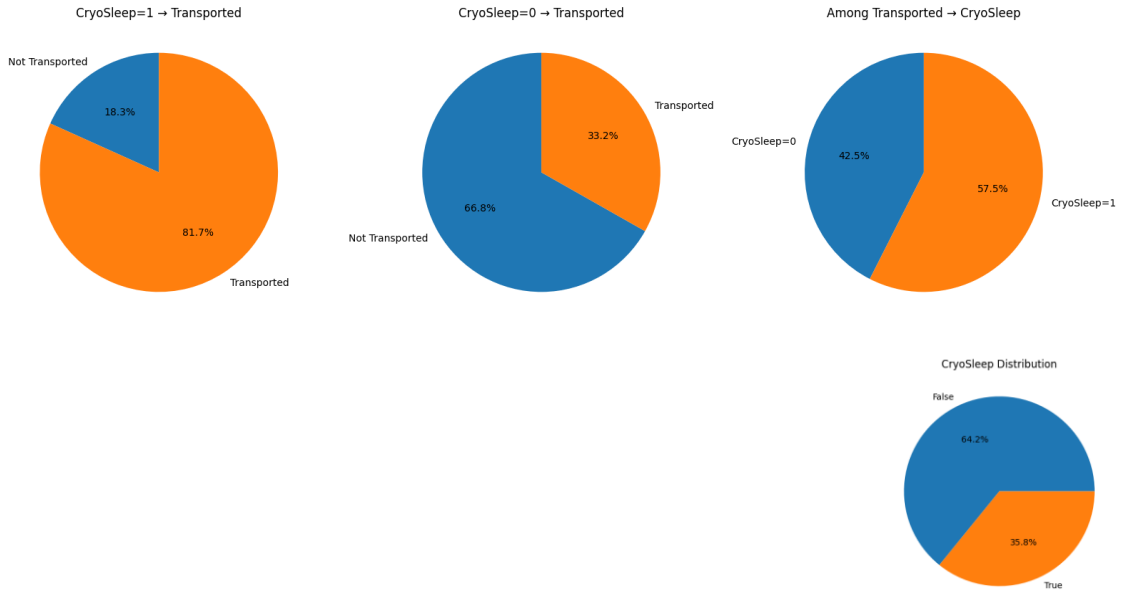
TotalSpend: עמודה שקובעת את סכום הבזבוזים הכולל של הנוסע (גם עמודה שיצרנו, מה שמראה שהבדיקות שביצענו בEDA היו נכונות ועזרו לנו להגיע לביצועי מסווג גבוהים יותר).



אפשר לראות מהגרף שאנשים שבזבזו פחות, היה להם יותר סיכוי לעבור טרנספורמציה.

בתהליך הEDA גילינו קשר ישיר בין הבזבוזים לבין ההחלטה אם לישון בטיסה, אז כנראה שילובים ביחד נותן למודל משקל יותר כבד לחלק הזה.

CryoSleep: האם הנוסע היה בשינה יזומה במהלך הטיסה או לא.



81 אחוז מהאנשים שישנו עברו טרנספורמציה, זהו אחוז גבוהה מאוד וכנראה זאת הסיבה שעמודה זאת קיבלה כלכך הרבה משקל.

לסיכום:

במהלך העבודה בוצע תהליך מלא של ניתוח נתונים ובניית מודלי סיווג לצורך חיזוי המשתנה Transported. תחילה בוצע שלב EDA מקיף שכלל ניתוח התפלגויות, בדיקת קשרים בין משתנים, טיפול בערכים חסרים, והנדסת תכונות מתאימה. שלב זה אפשר הבנה עמוקה של הנתונים וזיהוי מאפיינים משמעותיים כגון HomePlanet, CryoSleep ו־TotalSpend, אשר תרמו לשיפור תהליך הלמידה של המודל המנצח.

בהמשך נבנו ואומנו מספר מסווגים שונים: KNN, LDA, QDA, Gaussian Naive Bayes, Decision Tree, Random Forest, SVM, Logistic Regression, MLP ו־XGBoost. עבור כל מסווג בוצע תהליך שיטתי של בחירת Hyperparameters באמצעות ניסויים ו־Cross Validation, תוך שמירה על הפרדה מלאה בין קבוצת האימון לבין קבוצת הבדיקה. ביצועי המודלים הוערכו באמצעות מדדים שונים כגון Accuracy, Precision, Recall, F1-score ו־ROC-AUC, וכן באמצעות ניתוח גרפי שכלל מטריצות בלבול, עקומות ROC והשוואות בין ביצועי Train ו־Test.

מתוצאות הניסויים עולה כי מודל XGBoost השיג את הביצועים הטובים ביותר על קבוצת המבחן, עם איזון טוב בין דיוק גבוה לבין יכולת הכללה, תוך הימנעות יחסית מ־Overfitting למודלים אחרים. ניתוח חשיבות התכונות הדגיש כי המודל מצליח לנצל קשרים מורכבים ואינטראקציות בין משתנים, דבר שהוביל לשיפור בביצועים ביחס למסווגים הפשוטים יותר.

לסיכום, השילוב בין ניתוח נתונים מקדים, בחירה מושכלת של מודלים, וכיוונון Hyperparameters אפשר להגיע למסווג בעל ביצועים יציבים וסיכויי דיוק גבוהים על נתונים עתידיים לא מתויגים.